Assignment 2

Computer Networks (CS 456/656)

Winter 2026

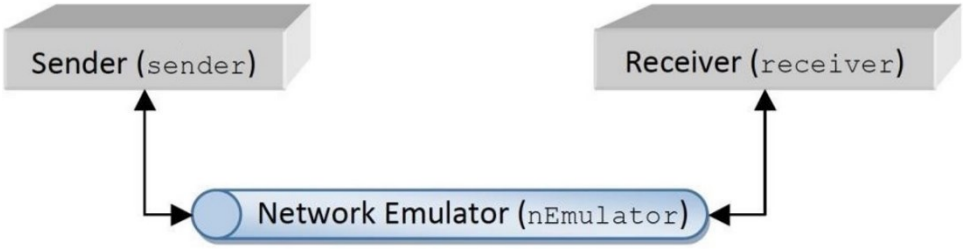
A network-assisted congestion-controlled pipelined RDT

Due Date: March 9th, 2026, at midnight (11:59 PM)

Work on this assignment is to be completed individually

Assignment Objective

The goal of this assignment is to implement a **network-assisted congestion-controlled, pipelined Reliable Data Transfer (RDT) protocol over UDP** to transfer a text file from one host to another across an unreliable network. The protocol should be able to handle packet loss, explicit congestion notification (ECN), packet reordering, and duplicate packets. For simplicity, your protocol is unidirectional, i.e., data will flow in one direction (from the sender to the receiver) and the acknowledgements (ACKs) in the opposite direction. To implement this protocol, you will write two programs: a sender and a receiver, with the specifications given below. To test your implementation, we will provide a third program, the network emulator, that will emulate an unreliable network link.



When the sender needs to send packets to the receiver, it sends them to the network emulator instead of sending them directly to the receiver. The network emulator then forwards the received packets to the receiver. However, it may randomly discard or reorder the received packets. The same scenario happens when the receiver sends ACKs to the sender.

Packet Format

All packets exchanged between the sender and the receiver should have the following structure:

```
integer type;           // 0: ACK, 1: Data, 2: EOT,
integer seqnum;         // Modulo 32
integer length;         // Length of the String variable 'data'
integer ecn;            // 0: Default, 1: Congestion mark set by
                        // network emulator on data packets
```

```
integer ce_count        // CE (congestion experienced) count:
                        // Set by the receiver to the total number of
                        // ECN-marked data packets received
String data;            // String with Max Length 500
```

Each integer field is a 4-byte unsigned integer in **network byte order**.

- The `type` field indicates the type of the packet. It is set to 0 if it is an ACK, 1 if it is a data packet, 2 if it is an end-of-transmission (EOT) packet (see the definition and use of EOT packets below).
- For data packets, `seqnum` *is* *the modulo 32 sequence number of the packet*. The sequence number of the first *data packet* should be zero. For ACK packets, `seqnum` is the sequence number of the *packet being acknowledged*.
- The `length` field specifies the number of characters carried in the data field. It should be in *the range of 0 to 500*. The `data` string should be exactly `length` bytes long. For EOT and ACK packets, `length` should be set to zero.
- The `ecn` field is used by the network emulator to indicate congestion. The network emulator sets its value to 1 when it is experiencing congestion, and to 0 otherwise.
- The `ce_count` field is a cumulative ECN-mark count used to enable fraction-based ECN feedback at the sender. For **data packets**, `ce_count` is unused and should be set to 0. For **ACK packets**, `ce_count` is set by the receiver to the cumulative number of ECN-marked data packets at the receiver.

A reference implementation of the packet format is provided to you as a Python 3 file named "packet.py". Please view the code at the end of the file for sample usage of the Packet class.

Network Emulator (`nEmulator`)

The network emulator is provided to you as a Python 3 program. When the emulator receives a data packet from the sender, it will discard it with the specified probability. Otherwise, it stores the packet in its buffer and later forwards it to the receiver with a random delay. The same behaviour applies to ACKs received from the receiver. An EOT packet from the sender is never discarded. It is forwarded to the receiver once it has forwarded all data packets awaiting in the buffer. An EOT packet from the receiver is also never discarded. It is forwarded to the sender once there are no more ACKs in the buffer.

In addition, the emulator simulates Explicit Congestion Notification (ECN) marking based on a target packet rate. Let R_t be the target packet rate (packets per second). The emulator measures the current incoming **data** packet rate R_{cur} over a fixed window of $W = 100$ ms. Specifically, every 100ms the emulator counts the number of data packets received from the sender during the previous 100ms window; if this count is C, then: $R_{cur} = \frac{C}{0.1}$. If $R_{cur} \leq R_t$, the ECN marking probability is set to 0. Otherwise, the emulator computes an ECN marking probability:

$$p_{ecn} = \min\left(1, \frac{R_{cur} - R_t}{R_t}\right)$$

For each forwarded **data** packet (that is not discarded), the emulator independently sets the packet's `ecn` field to 1 with probability p_{ecn}, and sets it to 0 otherwise. (ACK and EOT packets are not ECN-marked.)

To run `nEmulator`, you need to supply the following command line parameters in the given order:

- `<emulator's receiving UDP port number in the forward (sender) direction>`,
- `<receiver's network address>`,
- `<receiver's receiving UDP port number>`,
- `<emulator's receiving UDP port number in the backward (receiver) direction>`,
- `<sender's network address>`,
- `<sender's receiving UDP port number>`,
- `<packet discard probability>`,
- `<Target packet rate to prevent congestion. Range 10-100 (packets per sec)>`
- `<verbose-mode>` (Boolean: Set to 1, the network emulator will output its internal processing).

Receiver Program (`receiver`)

You should implement the receiver program, named as `receiver`, on a UNIX system. Its command line input includes the following:

- `<hostname for the network emulator>`,
- `<UDP port number used by the link emulator to receive ACKs from the receiver>`,
- `<UDP port number used by the receiver to receive data from the emulator>`, and
- `<name of the file into which the received data is written>` in the given order.

When receiving packets sent by the sender via the network emulator, it should execute the following:

- Check the sequence number of the packet.
- If the sequence number is the one that it is expecting:
  - If the packet is an EOT packet, send an EOT packet back and terminate the program. Otherwise, write the data from the packet to the output file. Then check if the packet with the next sequence number is in the buffer.
  - If the packet exists, remove the packet from the buffer, write the data of the packet to the output file, then repeat the previous step.

    - If the packet does not exist, send an ACK packet back to the sender with the `seqnum` equal to the `seqnum` of the last packet written to disk and set the expected `seqnum` to the `seqnum` of the missing packet.
- Otherwise, if the sequence number is not the one that it is expecting:
    - If the sequence number is within the next 10 sequence numbers, store the received packet in a buffer if the packet is not already stored.
    - In all other cases (e.g., duplicate/old packet), discard the received packet.
    - For both the cases above, send an ACK packet for the most recently received in-order packet.
- Congestion Notification
    - The receiver maintains a cumulative ECN-mark counter `ce_total`, initialized to 0. `ce_total` is incremented by 1 each time an in-order data packet is delivered (written to disk) whose `ecn` field equals 1 (including packets released from the buffer).
    - Whenever the receiver sends an ACK with `seqnum = n` (cumulative acknowledgement of all packets up to n), it must set the ACK's `ce_count` field equal to `ce_total`.

Once the receiver has received all data packets and an EOT from the sender, it should send an EOT packet then exit.

Output

The receiver program is also required to generate a log file, named *arrival.log*. The file *arrival.log* should record the sequence numbers and ecn fields of all the data packets that the receiver receives during the entire period of transmission. The sequence number for EOT packets should be logged as "EOT" and the ECN field need not be logged. *The format for the log file is two numbers per line (**no timestamp**). You must follow the format to avoid losing marks.* The format is: `seq ecn \n`

Sender Program (`sender`)

You should implement a sender program, named `sender`. Its command line input includes the following:

- `<host address of the network emulator>`,
- `<UDP port number used by the emulator to receive data from the sender>`,
- `<UDP port number used by the sender to receive ACKs from the emulator>`, and
- `<name of the file to be transferred>` in the given order.

Upon execution, the sender program goes through 2 stages: data transmission and connection termination. In the data transmission stage, the sender program reads data from the specified file and sends it to the receiver over the network emulator using the congestion-controlled RDT protocol. The

initial window size should be set to $N = 1$ packet. The first data packet is transmitted with `seqnum=0`, the second packet is transmitted with `seqnum=1`, and so on. After the packet with `seqnum=31` is transmitted, the next packet is transmitted with `seqnum=0`. After all content in the file has been successfully transmitted to the receiver (and all corresponding ACKs have been received), the sender transitions to the connection-termination stage. In this stage, the sender sends an EOT packet to the receiver. The sender can close its socket and exit only after it has received ACKs for all data packets, it has sent an EOT to the receiver and it has received an EOT acknowledgement. To keep the project simple, you can assume that the end-of-transmission packet never gets lost in the network.

To ensure reliable transmission and congestion control in the data transmission stage, your program should implement the congestion controlled pipelined RDT protocol as follows:

If the sender has a packet to send, it first checks to see if the window is full, that is, whether there are N outstanding, unacknowledged packets. If the window is full, the sender will try sending the packet later. If the window is not full, the packet is sent, the appropriate variables are updated, and a packet loss timer is started if it has not already been started. For tracking packet loss, the sender will use a single timer (packet loss timer) associated with the oldest transmitted-but-not-yet-acknowledged packet. If a new ACK is not received before the timeout, the sender assumes the packet to be lost.

When the sender receives an acknowledgement packet with `seqnum = n`, the ACK will be taken to be a cumulative acknowledgement, indicating that all packets with a sequence number up to and including `n` have been correctly received at the receiver. If a new ACK (not a duplicate ACK) is received while there are still additional transmitted but not yet acknowledged packets, the timer is restarted. If there are no outstanding packets, the timer is stopped.

The sender reacts to network congestion by changing its window size N: N is increased when packets are successfully received and N is decreased when the sender detects congestion. To this end, the sender maintains a real-valued congestion window variable cwnd and sets the integer window size N to the integer value of cwnd, i.e., $N = \lfloor cwnd \rfloor$, with $1 \leq N \leq 10$.

Additive increase (on each new ACK)

If a new (non-duplicate) ACK is received, the sender performs an additive increase to cwnd as:

$$cwnd = cwnd + \frac{1}{cwnd}$$

After this update, the sender sets the window size as: $N = \min(10, \max(1, \lfloor cwnd \rfloor))$

Congestion detection and response

The sender detects network congestion in two ways.

1) Packet drop (timeout): If a timeout of 300ms occurs, the sender sets cwnd = 1 (and therefore $\mathrm{N} = 1$) and retransmits only the oldest transmitted-but-not-yet-acknowledged packet. The timer is then reset.

2) ECN feedback using `ce_count` (fraction-based): Each ACK packet contains a cumulative ECN-mark count `ce_count`, which is the total number of ECN-marked data packets delivered at the receiver so far. The sender uses this to estimate the **fraction of marked packets $\mathbf{F}$** in the last RTT (assume RTT ≈ 100ms). For this,

- For this, the sender maintains a 2nd timer (RTT timer) and the following variables:
  - α (congestion level estimate, initialized to 0 at the start of program)
  - `prev_ce_count` (initialized to 0 at start of program)
  - `acked_in_rtt` (initialized to 0 at start of RTT)
  - `marked_in_rtt` (initialized to 0 at start of RTT)
  - *Start the initial RTT timer at the beginning of the program*
- When a **new (non-duplicate)** ACK with cumulative `seqnum=n` is received, the sender updates:
  - `acked_in_rtt += (seqnum - prev_seqnum + 32) mod 32 (e.g., ACK seqnum=2, prev seqnum=28, acked_in_rtt+=6)`
  - `marked_in_rtt += ce_count - prev_ce_count`
  - `prev_ce_count = ce_count`
  - `prev_seqnum = seqnum`

At the end of every RTT (every RTT timer tick), the sender computes the fraction of marked packets received at receiver as:

$$F = \frac{marked_in_rtt}{acked_in_rtt} \text{(only if } acked_\text{in_rtt} > 0\text{)}$$

and updates its estimate of congestion level α using an exponentially weighted moving average:

$$\alpha = (1 - 0.0625)\alpha + 0.0625 \cdot F$$

After this update, the sender performs ECN-based multiplicative decrease in cwnd as:

$$cwnd = cwnd \cdot (1 - \alpha/2)$$

and updates its window size: $N = \min(10, \max(1, \lfloor cwnd \rfloor))$

Finally, after applying the update, the sender resets the RTT timer, and the following variables:

- `acked_in_rtt = 0`
- `marked_in_rtt = 0`

Thus, successful acknowledgements gradually increase the window size, packet loss causes a strong reduction of the window to 1, and ECN marking causes a proportional reduction based on the estimated fraction of marked packets over each RTT.

Output

For both testing and grading purposes, your *sender* program should be able to *generate three log files*, named *seqnum.log*, *ack.log*, and *N.log*. Whenever a packet is sent, its sequence number should be recorded in *seqnum.log*. The file *ack.log* should record the sequence numbers of all the ACK packets and their ce_count fields that the sender receives over the entire transmission period. For EOT packets, the sequence number written to the file should be "EOT". *N.log* should record the initial value of N, as well as every time the value of N is changed. *The format for these log files is one timestamp, space, and one sequence number per line*.

Timestamps are recorded as "t=X", where X is the timestamp of the current action. The timestamp is a number that is incremented by one at every new **event** (i.e., a new packet to be sent, receiving an ACK, or timeout). The timestamp t=0 is reserved for initialization, and the only event that happens during this initialization is the window size N initializing to 1. The subsequent data transmission beings at t=1.

N.log will have, t=0 1 as the first line in the log. For the first data packet, your program should write t=1 0 in *seqnum.log* for packet #0 sent at t=1. If an EOT is sent by the sender at t=105, then the log should record t=105 EOT. Similarly, if the sender receives EOT from the receiver at t=106, then the ack.log should record t=106 EOT <`ce_count` value>.

Be careful: *some actions are executed at the same timestamp*. For example, if a timeout occurs at t=T, there should be an entry t=T in *seqnum.log* for the retransmission, as well as in *N.log* for (re)setting N to 1. You must follow this format to avoid losing points.

Hints

- *You must ensure your programs run in the CS student environment*
- Experiment with network delay values and sender time-out to understand the performance of the protocol.
- Please ensure that your implementation works even if the three programs run on separate machines within the CS student environment.

Example Execution

1. On the host **host1**: `nEmulator 9991 host2 9994 9993 host3 9992 0.05 10 1`
2. On the host **host2**: `receiver host1 9993 9994 <output File>`
3. On the host **host3**: `sender host1 9991 9992 <input file>`

Note: Use different ports than the ones listed here, as these might be in-use by your classmates.

Procedures

Due Date

The assignment is due on **March 9th, 2026, at midnight (11:59 PM)**.
Late submission policy: 10% penalty for every late day, up to 3 late days. Submissions are not accepted beyond 3 late days.

Hand-in Instructions

Submit all your files in a single compressed file (.zip, .tar, etc.) using LEARN. The filename should include your username and/or student ID.

You must hand in the following files / documents:

- *Source code* files as a zip. Please make sure all files are present in the root dir (no folders). The name of the zip-file should be your quest username.
- *Makefile (if applicable)*: your code **must** compile and link cleanly by typing "*make*" or "*gmake*".
- *README* file: this file ***must*** contain instructions on how to run your program, which CS student environment machines your program was built and tested on, and what version of *make* and *compilers* you are using (if applicable).

Your implementation will be tested on the machines available in the **CS student environment**.

Documentation

Since there is no external documentation required for this assignment, you are expected to have a reasonable amount of internal code documentation (to help the markers read your code).

<u>You **will** lose points if your code is unreadable, sloppy, and inefficient</u>.